

TAREA: DISEÑO DE FRAGMENTACIÓN

Base de datos distribuida con AdventureWorks2019

Alumna

Karina Michelle Peña Armas

Materia

Base de Datos Distribuidas

Ejercicio seleccionado: Fragmentación Horizontal Primaria de Sales.Customer

□

Introducción

La fragmentación es una técnica utilizada en bases de datos distribuidas para dividir una relación en partes más pequeñas que pueden almacenarse o administrarse en distintos nodos. En este trabajo se propone un diseño de fragmentación para la base de datos AdventureWorks2019, utilizando como tabla propietaria Sales.Customer y el atributo CustomerID como criterio de fragmentación.

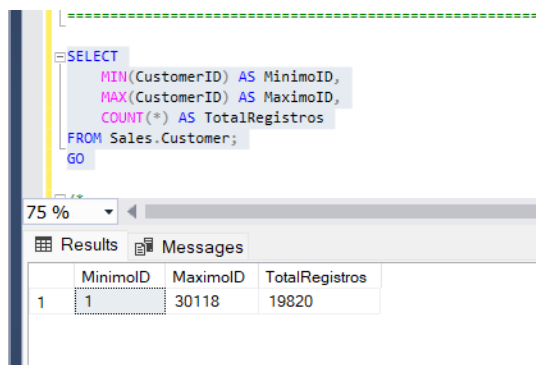
El diseño incluye fragmentación horizontal primaria, aplicación de predicados mediante COM_MIN, fragmentación horizontal derivada por medio de semi-join y la validación de las propiedades de completitud, reconstrucción y disyunción.

1. Selección del ejercicio

Se seleccionó el Ejercicio 3: distribuir la cartera de clientes en cinco nodos, de forma que cada fragmento contenga un subconjunto de los clientes. La tabla propietaria es Sales.Customer y el atributo utilizado para definir los rangos es CustomerID.

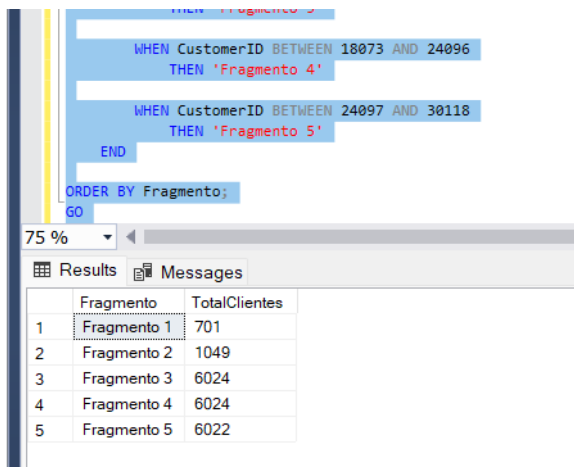
```
USE AdventureWorks2019;  
GO
```

```
SELECT  
    MIN(CustomerID) AS MinimoID,  
    MAX(CustomerID) AS MaximoID,  
    COUNT(*) AS TotalRegistros  
FROM Sales.Customer;
```



	MinimoID	MaximoID	TotalRegistros
1	1	30118	19820

A partir del intervalo de CustomerID se definieron cinco rangos. Los rangos tienen amplitudes numéricas semejantes, aunque la cantidad real de registros no es uniforme debido a que no todos los identificadores del intervalo existen en la tabla.



2. Segmento del grafo relacional

Para obtener fragmentos derivados se consideraron las tablas relacionadas con el cliente y sus órdenes de venta. Sales.Customer es la tabla propietaria; Sales.SalesOrderHeader es miembro respecto de Customer y Sales.SalesOrderDetail es miembro respecto de SalesOrderHeader.

```

Sales.Customer
|
| CustomerID
v
Sales.SalesOrderHeader
|
| SalesOrderID
v
Sales.SalesOrderDetail

```

Las relaciones utilizadas son Customer.CustomerID = SalesOrderHeader.CustomerID y SalesOrderHeader.SalesOrderID = SalesOrderDetail.SalesOrderID.

3. Aplicación del algoritmo COM_MIN

Para la fragmentación horizontal primaria se parte de predicados simples que representan los puntos de corte del atributo CustomerID. El objetivo de COM_MIN es conservar un conjunto de predicados completo y mínimo para distinguir los fragmentos requeridos.

```

PR = {
p1: CustomerID < 6025,
p2: CustomerID < 12049,
p3: CustomerID < 18073,
p4: CustomerID < 24097
}

```

Los cuatro predicados son necesarios para distinguir cinco regiones. Si se elimina cualquiera de los puntos de corte, dos intervalos adyacentes dejan de poder diferenciarse. Por ello, para las consultas consideradas en este diseño, el conjunto mínimo utilizado se mantiene como PR' = {p1, p2, p3, p4}.

```

m1: CustomerID < 6025
m2: CustomerID >= 6025 AND CustomerID < 12049
m3: CustomerID >= 12049 AND CustomerID < 18073
m4: CustomerID >= 18073 AND CustomerID < 24097
m5: CustomerID >= 24097

```

4. Generación de fragmentos primarios y derivados

4.1 Fragmentación horizontal primaria

Los cinco fragmentos de Sales.Customer se crearon con SELECT INTO y condiciones de rango sobre CustomerID.

```

SELECT * INTO dbo.Customer_1
FROM Sales.Customer
WHERE CustomerID BETWEEN 1 AND 6024;

SELECT * INTO dbo.Customer_2
FROM Sales.Customer
WHERE CustomerID BETWEEN 6025 AND 12048;

SELECT * INTO dbo.Customer_3
FROM Sales.Customer
WHERE CustomerID BETWEEN 12049 AND 18072;

SELECT * INTO dbo.Customer_4
FROM Sales.Customer
WHERE CustomerID BETWEEN 18073 AND 24096;

SELECT * INTO dbo.Customer_5
FROM Sales.Customer
WHERE CustomerID BETWEEN 24097 AND 30118;

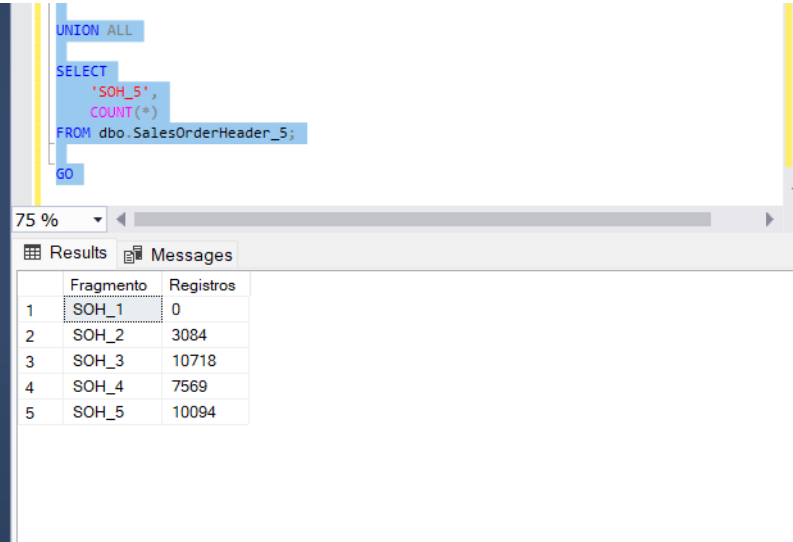
```

4.2 Fragmentación derivada de SalesOrderHeader

Para cada fragmento de Customer se seleccionaron únicamente las órdenes relacionadas con sus clientes. Esta operación corresponde a la fragmentación horizontal derivada mediante semi-join.

```
SELECT SOH.*
INTO dbo.SalesOrderHeader_1
FROM Sales.SalesOrderHeader AS SOH
INNER JOIN dbo.Customer_1 AS C
    ON SOH.CustomerID = C.CustomerID;

/* El mismo procedimiento se aplica a los fragmentos 2 a 5. */
```



	Fragmento	Registros
1	SOH_1	0
2	SOH_2	3084
3	SOH_3	10718
4	SOH_4	7569
5	SOH_5	10094

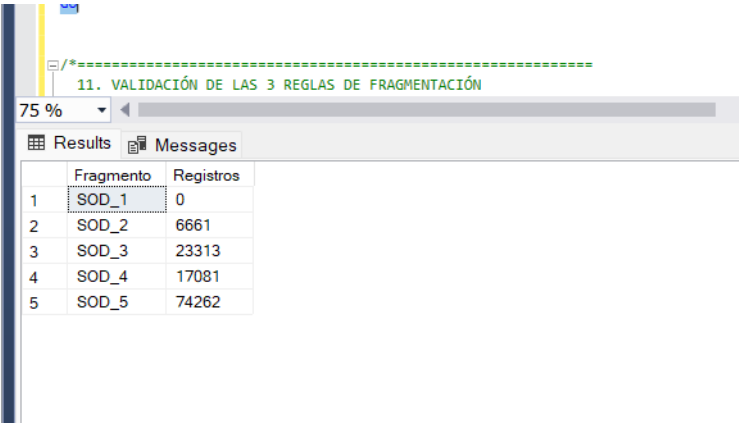
El primer fragmento derivado contiene cero órdenes. Esto no representa un error: los clientes que pertenecen al primer rango no tienen registros relacionados en Sales.SalesOrderHeader.

4.3 Fragmentación derivada de SalesOrderDetail

En un segundo nivel, Sales.SalesOrderDetail se fragmentó usando SalesOrderID para conservar únicamente los detalles pertenecientes a las órdenes de cada fragmento.

```
SELECT SOD.*
INTO dbo.SalesOrderDetail_1
FROM Sales.SalesOrderDetail AS SOD
INNER JOIN dbo.SalesOrderHeader_1 AS SOH
    ON SOD.SalesOrderID = SOH.SalesOrderID;

/* El mismo procedimiento se aplica a los fragmentos 2 a 5. */
```



	Fragmento	Registros
1	SOD_1	0
2	SOD_2	6661
3	SOD_3	23313
4	SOD_4	17081
5	SOD_5	74262

5. Validación de las reglas de fragmentación

5.1 Completitud

La completitud exige que cada registro de la relación original se encuentre en alguno de los fragmentos. Sales.Customer contiene 19,820 registros y la suma de Customer_1 a Customer_5 también es 19,820.

```

SELECT
    COUNT(*) AS TotalOriginal
FROM Sales.Customer;

SELECT
    (SELECT COUNT(*) FROM dbo.Customer_1)
    + (SELECT COUNT(*) FROM dbo.Customer_2)
    + (SELECT COUNT(*) FROM dbo.Customer_3)
    + (SELECT COUNT(*) FROM dbo.Customer_4)
    + (SELECT COUNT(*) FROM dbo.Customer_5)
    AS TotalFragmentos;

GO

```

75 %

Results Messages

	TotalOriginal
1	19820

	TotalFragmentos
1	19820

Por lo tanto, se cumple la propiedad de completitud.

5.2 Reconstrucción

Al tratarse de fragmentación horizontal, la relación original puede reconstruirse mediante UNION ALL de los cinco fragmentos. La consulta de reconstrucción devolvió nuevamente 19,820 registros.

```

SELECT COUNT(*) AS TotalReconstruido
FROM (
    SELECT * FROM dbo.Customer_1
    UNION ALL SELECT * FROM dbo.Customer_2
    UNION ALL SELECT * FROM dbo.Customer_3
    UNION ALL SELECT * FROM dbo.Customer_4
    UNION ALL SELECT * FROM dbo.Customer_5
) AS Reconstruccion;

```

El total reconstruido coincide con la tabla original, por lo que se cumple la propiedad de reconstrucción.

75 %

Results Messages

	TotalReconstruido
1	19820

5.3 Disyunción

La disyunción establece que un registro no debe pertenecer simultáneamente a dos fragmentos horizontales. Los rangos utilizados no se superponen. La consulta de validación devolvió 0 filas duplicadas.

```

SELECT CustomerID, COUNT(*) AS Veces
FROM (
    SELECT CustomerID FROM dbo.Customer_1
    UNION ALL SELECT CustomerID FROM dbo.Customer_2
    UNION ALL SELECT CustomerID FROM dbo.Customer_3
    UNION ALL SELECT CustomerID FROM dbo.Customer_4
    UNION ALL SELECT CustomerID FROM dbo.Customer_5
) AS Fragmentos
GROUP BY CustomerID
HAVING COUNT(*) > 1;

```

Al obtener 0 filas, ningún CustomerID aparece en más de un fragmento y se cumple la propiedad de disyunción.

5.4 Comprobación de los fragmentos derivados

También se verificaron las tablas derivadas. La suma de los cinco fragmentos de SalesOrderHeader fue 31,465 registros, igual al total de la tabla original. Para SalesOrderDetail la suma fue 121,317 registros, también igual al total original.

6. Conclusión

En esta tarea realicé la fragmentación de la tabla Sales.Customer en cinco partes utilizando el CustomerID. Después generé los fragmentos relacionados de SalesOrderHeader y SalesOrderDetail mediante semi-join.

Con las pruebas comprobé que se cumplen las tres reglas de fragmentación: completitud, reconstrucción y

disyunción. Aunque los datos no quedaron distribuidos de manera uniforme, la fragmentación funciona correctamente y permite recuperar todos los datos originales.